13. 실전 프로젝트

이 장에서는 Hermes Agent를 활용한 실전 프로젝트를 구축합니다. 앞에서 배운 도구, 메모리, 스킬, MCP, 크론 등의 기능을 종합적으로 활용하여 실제로 쓸 수 있는 시스템을 만듭니다. 각 프로젝트는 독립적이므로 관심 있는 것부터 시작해도 됩니다.

이 장에서 다루는 내용

- 프로젝트 코드 분석 에이전트
- 일일 뉴스 브리핑 자동화
- 팀 협업 봇 구축
- Mac Mini 홈 인텔리전스 허브

13-1. 프로젝트 코드 분석 에이전트

> ⚠️ ***부분 검증**: 소스 대조가 가능한 서술(AGENTS.md 자동 로드, 컨텍스트 파일 2만 자 한도, 스킬 저장 경로)은 v0.16.0 기준으로 확인했습니다. 본문의 분석 요청문과 리포트는 실제 출력이 아니라 작성 예시이며, 분석 품질은 사용한 모델과 대상 저장소에 따라 달라집니다.*

- 프로젝트: 코드 분석 에이전트
- 이 문서에서 다루는 내용
- 사전 준비
- AGENTS.md 설정
- 실습: 저장소 분석
  - 기본 분석 요청
  - 심화 분석 요청
- 리포트 출력 예시
- 반복 작업을 스킬로 만들기
- 활용 아이디어

프로젝트: 코드 분석 에이전트

이 프로젝트에서는 Hermes Agent를 사용하여 코드 저장소를 자동으로 분석하는 에이전트를 구성합니다. 별도 설정 없이 바로 시작할 수 있는 가장 간단한 프로젝트입니다.

이 문서에서 다루는 내용

이 프로젝트는 다음을 다룹니다.

- Git 저장소를 클론해 구조·의존성·아키텍처를 분석
- 결과를 마크다운 리포트(`report.md`)로 정리
- 같은 분석을 반복하면 스킬(`repo-analysis`)로 굳혀 다음부터는 한 줄로 실행

사전 준비

- Git 설치 여부 확인: `git --version`
- Hermes Agent를 실행할 수 있어야 합니다.

- `AGENTS.md`를 저장할 작업 디렉터리를 준비합니다. 예: `~/analysis-workspace/`. 이 디렉터리에 분석 대상 저장소를 클론합니다.

AGENTS.md 설정

작업 디렉터리에 `AGENTS.md`를 둡니다. `AGENTS.md`가 프로젝트 디렉터리 안에 있으면 Hermes Agent가 자동으로 읽으므로, 따로 설치할 필요는 없습니다. 같은 분석을 다른 머신에서도 반복하려면 이 파일을 함께 복사하거나, 「프로파일」 절에서 설명하는 방식으로 설정을 묶어 관리할 수 있습니다.

```
# AGENTS.md

## 코드 분석 규칙

### 분석 순서
1. 파일 구조를 먼저 파악할 것 (tree 또는 ls -R)
2. 의존성 파일 확인: package.json, requirements.txt, Cargo.toml, go.mod 등
3. README.md가 있으면 읽을 것
4. 진입점(entry point)을 찾을 것
5. 테스트 구조를 파악할 것

### 출력 규칙
- 결과는 마크다운 형식으로 report.md에 저장
- 각 섹션에 구체적인 파일 경로를 포함
- 의존성은 표(table) 형식으로 정리
- 개선 제안을 최소 3개 포함
```

실습: 저장소 분석

기본 분석 요청

```
> https://github.com/NousResearch/hermes-agent 저장소를 클론해서 분석해줘.
> 다음 항목을 포함하는 리포트를 만들어줘:
> 1. 프로젝트 구조 (디렉터리 트리)
> 2. 사용된 프로그래밍 언어 비율
> 3. 주요 의존성 목록
> 4. 진입점(entry point) 파일
> 5. 테스트 구조
```

에이전트가 터미널 도구로 저장소를 클론하고, 파일을 읽고, 분석 결과를 리포트로 정리하는 과정을 관찰합니다.

심화 분석 요청

기본 분석이 끝나면 다음과 같은 추가 질문을 할 수 있습니다.

```
> 이 프로젝트에서 보안 관련 코드를 찾아줘
> 에러 처리 패턴이 일관적인지 확인해줘
> 테스트 커버리지가 부족한 모듈을 찾아줘
```

리포트 출력 예시

에이전트가 생성하는 `report.md` 는 대략 이런 구조입니다.

```
## 프로젝트 분석 리포트: hermes-agent

### 1. 프로젝트 구조
hermes/
├── agent/        # 에이전트 코어 로직
├── tools/        # 내장 도구 구현
├── gateway/      # 메시징 게이트웨이
└── config/       # 설정 관리

### 2. 기술 스택
| 항목 | 값 |
|------|-----|
| 언어 | Python (91.4%) |
| 패키지 관리 | pip / pyproject.toml |
| LLM | Ollama / API |

### 3. 주요 의존성
| 패키지 | 용도 |
|--------|------|
| litellm | 다중 LLM 백엔드 통합 |
| prompt-toolkit | CLI 인터페이스 |
| pydantic | 데이터 검증 |
...

### 4. 개선 제안
- ...
```

분석 품질 높이기

AGENTS.md에 분석 규칙을 구체적으로 작성할수록 리포트 품질이 올라갑니다. 예를 들어 "보안 취약점도 체크할 것", "라이선스 호환성을 확인할 것" 같은 규칙을 추가해 보세요. 단, 컨텍스트 파일은 소스별로 **최대 20,000자**까지만 주입되고 초과분은 절단되므로(6장 참조), 핵심 규칙 위주로 간결하게 작성하는 것이 좋습니다.

반복 작업을 스킬로 만들기

같은 작업을 여러 저장소에서 반복한다면, 이 분석 절차를 `repo-analysis` 스킬로 저장해 둘 수 있습니다.

```
> 이 분석 작업을 스킬로 만들어줘. 이름은 "repo-analysis"로.
```

에이전트가 만든 스킬은 `~/.hermes/skills/repo-analysis/` 디렉터리 아래 `SKILL.md` 파일로 저장되며, 다음과 같은 구조입니다.

```
---
name: repo-analysis
description: Git 저장소를 클론하여 구조, 의존성, 아키텍처를 분석하고 리포트를 생성합니다.
---

## 실행 절차
1. 대상 저장소를 임시 디렉터리에 클론
2. 디렉터리 구조 파악 (tree -L 3)
3. 의존성 파일 분석
4. 진입점 탐색
5. 테스트 구조 확인
6. report.md 생성

## 출력 형식
마크다운 리포트: 구조, 스택, 의존성, 진입점, 테스트, 개선 제안
```

다음부터는 스킬 이름만 입력해 분석할 수 있습니다.

```
> repo-analysis https://github.com/some/project
```

활용 아이디어

- **PR 리뷰 보조**: 변경된 파일만 분석하여 리뷰 코멘트 초안 작성

- **기술 부채 추적**: 주기적으로 분석하여 코드 품질 변화 모니터링
- **온보딩 문서 생성**: 새 팀원을 위한 프로젝트 구조 설명서 자동 생성
- **의존성 감사**: 오래된 패키지, 보안 취약점이 있는 의존성 탐지

13-2. 일일 뉴스 브리핑 자동화

> ⚠️ ***부분 검증**: 이 페이지의 명령·경로·함수(`hermes gateway` · `hermes cron list` · `/sethome` · `parse_schedule()` · 웹 검색 구성)는 v0.16.0 소스로 재확인했습니다. 다만 "자연어 한 줄 → 크론 등록 → 정시 발송 → 중복 방지"로 이어지는 종단 동작은 LLM 판단과 외부 검색 API에 의존하므로, 한 가지 구성 예시로 보시기 바랍니다.*

- 프로젝트: 일일 뉴스 브리핑 자동화
- 목표
- 사전 준비
- 설정 단계
  - 1. 웹 검색 준비
  - 2. 메시징 게이트웨이 설정
  - 3. 크론 작업 등록
- 브리핑 메시지 예시
- 여러 주제 브리핑 설정
- 고급: 키워드 감지 즉시 알림
- 확장 아이디어
- 문제 해결
  - 검색 결과가 없을 때
  - 크론이 실행되지 않을 때

프로젝트: 일일 뉴스 브리핑 자동화

이 프로젝트에서는 크론과 메시징을 결합하여, 매일 아침 뉴스를 요약해서 메시징 앱으로 보내는 시스템을 구축합니다.

목표

- 매일 지정된 시간에 특정 주제의 뉴스를 검색
- 결과를 요약하여 마크다운으로 정리
- Telegram 또는 Discord로 전송

사전 준비

- Telegram 봇 설정 완료 (9장 참조)

- 웹 검색 도구 준비 (4장 참조)
- 크론 기능 이해 (10장 참조)

설정 단계

1. 웹 검색 준비

뉴스 검색에는 Hermes에 **내장된** `web_search` **도구**를 씁니다. 별도 MCP 서버를 붙일 필요 없이 바로 쓸 수 있고, 백엔드로 Brave Search·Tavily·SearXNG·DuckDuckGo 등을 고를 수 있습니다(4장 참조). 예를 들어 Brave Search를 쓰려면 API 키를 환경 변수에 넣습니다.

```
# ~/.hermes/.env
BRAVE_SEARCH_API_KEY=your_api_key
```

무료 검색 API

Brave Search API는 월 2,000회까지 무료라, 일일 브리핑 용도로는 충분합니다.

2. 메시징 게이트웨이 설정

Telegram 봇을 설정합니다 (9장 참조).

```
# 게이트웨이 실행
hermes gateway
```

Telegram에서 봇과 대화를 시작한 뒤, 크론 결과를 받을 채널을 홈 채널로 지정합니다.

```
/sethome
```

3. 크론 작업 등록

자연어로 크론 작업을 등록합니다.

```
> 매일 오전 8시에 "AI agent framework" 관련 뉴스를 검색하고,
> 한국어로 3줄 요약을 만들어서 Telegram으로 보내줘
```

크론 시스템에 별도의 자연어 파서가 있는 것이 아닙니다. **LLM이 자연어를 해석**하여 `parse_schedule()` 함수가 받아들이는 구조화된 형식(크론 표현식, 기간 문자열, ISO 타임스탬프)으로 변환합니다(10장 참조). 위 요청은 대략 다음과 같은 크론 작업으로 등록됩니다.

```
스케줄: 0 8 * * *    (매일 오전 8시, 크론 표현식)
작업: brave-search MCP로 "AI agent framework" 최신 뉴스 검색 →
      상위 5개 기사 선별 → 한국어 3줄 요약 → Telegram 전송 (검색은 내장 web_search 도구 사용)
```

브리핑 메시지 예시

Telegram으로 수신되는 메시지는 다음과 같은 형태입니다.

```
🗞️ AI Agent 뉴스 브리핑 (2025-01-15)

1. Hermes Agent v2.1 릴리스
   - 새로운 MCP 프로토콜 지원 추가
   - 스킬 허브에서 커뮤니티 스킬 설치 가능
   - 성능 30% 개선

2. OpenAI, 새로운 에이전트 프레임워크 발표
   - Function calling 기반 에이전트 SDK 공개
   - 멀티 에이전트 오케스트레이션 지원
   - Python/TypeScript SDK 동시 출시

3. LangChain vs CrewAI 벤치마크 비교
   - 실제 업무 시나리오 5개로 테스트
   - CrewAI가 멀티 에이전트 작업에서 우세
   - LangChain이 단일 에이전트에서 더 빠름
```

여러 주제 브리핑 설정

주제별로 다른 시간에 브리핑을 설정할 수 있습니다.

```
> 매일 오전 8시: AI/LLM 관련 뉴스 브리핑
> 매일 오전 8시 30분: 한국 주식시장 주요 뉴스 브리핑
> 매주 금요일 오후 6시: 이번 주 오픈소스 트렌드 주간 요약
```

고급: 키워드 감지 즉시 알림

특정 키워드가 감지되면 즉시 알림을 보내는 설정도 가능합니다.

```
> 30분마다 "hermes agent" 키워드로 뉴스를 검색하고,
> 새로운 기사가 발견되면 즉시 Telegram으로 알려줘.
> 이전에 보낸 기사는 중복 전송하지 마.
```

이 경우 에이전트는 메모리를 활용하여 이미 전송한 기사 URL을 기록하고, 중복을 방지합니다.

> **메모리 활용**
>
> 크론 작업에서 메모리를 활용하면 상태를 유지할 수 있습니다. "이전에 전송한 기사 목록을 메모리에 저장해줘"라고 지시하면, 에이전트가 `~/.hermes/memories/` 아래에 상태 파일을 관리합니다.

확장 아이디어

- **RSS 피드 모니터링**: 특정 블로그나 사이트의 RSS를 주기적으로 체크
- **주간 종합 리포트**: 일일 브리핑을 모아 주간 트렌드 분석
- **멀티 채널 전송**: 같은 브리핑을 Telegram + Discord에 동시 전송
- **감성 분석**: 뉴스의 긍정/부정 톤을 분석하여 시장 분위기 파악

문제 해결

검색 결과가 없을 때

검색 키워드가 너무 구체적이면 결과가 없을 수 있습니다. 키워드를 넓게 설정하세요.

```
# 너무 구체적
"hermes agent v2.1 release notes"

# 적절한 범위
"AI agent framework"
```

크론이 실행되지 않을 때

- `hermes gateway`가 실행 중인지 확인
- `hermes cron list`로 등록된 작업 확인
- 시간대(timezone) 설정이 올바른지 확인

13-3. 팀 협업 봇 구축

> ⚠️ ***부분 검증**: 이 페이지의 환경변수·CLI 명령·설정 키는 v0.16.0 소스 코드와 대조해 확인했습니다. 봇 대화 예시는 동작 설명을 위한 가상 시나리오이며, Discord·Slack 앱 등록과 토큰이 필요한 실제 구동은 검증 범위에 포함하지 않았습니다.*

- 프로젝트: 팀 협업 봇 구축
- 목표
- 아키텍처
- 설정
  - 1. Discord 봇 설정
  - 2. 사용자 인증
  - 3. 프로젝트 규칙 (AGENTS.md)
  - 4. 팀 스킬 생성
  - 5. 퍼스널리티 설정 (SOUL.md)
  - 6. 게이트웨이 실행
- 팀 사용 시나리오
  - 코드 리뷰 요청
  - 문서 검색
  - 배포 전 점검
- 보안 고려사항
- 운영 팁
  - 로그 모니터링
  - 스킬 버전 관리
  - 사용량 관리

프로젝트: 팀 협업 봇 구축

이 프로젝트에서는 Hermes Agent를 팀이 공유하는 Discord/Slack 봇으로 운영하는 방법을 다룹니다. 가장 복잡한 프로젝트이지만, 팀 생산성을 크게 높일 수 있습니다.

목표

- 팀원들이 Discord에서 에이전트에게 질문할 수 있는 환경 구성

- DM 페어링으로 인증된 팀원만 접근 허용
- AGENTS.md로 프로젝트 규칙을 에이전트에게 전달
- 공용 스킬로 팀 워크플로 자동화

아키텍처

```
팀원 (Discord) → Hermes Gateway → 에이전트 → 도구 실행
                                      ↓
                                 스킬/메모리
```

팀원은 Discord DM으로 에이전트와 대화합니다. 게이트웨이가 메시지를 중계하고, 에이전트가 도구를 실행하여 결과를 반환합니다.

설정

1. Discord 봇 설정

Discord Developer Portal에서 봇을 생성합니다 (9장 참조). **Message Content Intent**를 반드시 활성화해야 합니다. 이 설정이 꺼져 있으면 봇이 메시지 내용을 읽을 수 없습니다.

```
# .env
DISCORD_BOT_TOKEN=your_token
```

필요한 봇 권한:

- Send Messages
- Read Message History
- Manage Messages (선택)

2. 사용자 인증

`DISCORD_ALLOWED_USERS` 를 설정하지 않으면 **모든 사용자의 요청이 거부**됩니다. 보안을 위해 기본값이 "전부 차단"이므로, 반드시 허용할 팀원의 Discord 사용자 ID를 명시해야 합니다(9장 참조).

```
# config.yaml
discord:
  allowed_users:
    - "팀원1_discord_id"
    - "팀원2_discord_id"

# 또는 DM 페어링 사용 (새 팀원이 자주 추가될 때 편리)
unauthorized_dm_behavior: pair
```

DM 페어링 vs 허용 목록

팀 규모가 작고 고정적이면 `allowed_users`로 직접 관리하는 것이 안전합니다. 팀원이 자주 바뀌는 환경에서는 `pair` 모드가 편리하지만, 페어링 코드를 팀 내부 채널로만 공유하세요.

3. 프로젝트 규칙 (AGENTS.md)

팀의 컨벤션과 업무 규칙을 정의합니다.

```
# AGENTS.md

## 팀 프로젝트 규칙

### 기본 원칙
- 응답은 한국어로
- 코드 블록에는 항상 언어를 명시
- 불확실한 정보는 "확인 필요"라고 표시

### 코드 리뷰
- 보안 취약점 체크 (SQL 인젝션, XSS 등)
- 성능 이슈 지적
- 코딩 컨벤션 준수 여부 확인

### 안전 규칙
- DB 변경 작업은 반드시 사용자 확인 후 실행
- 프로덕션 서버 접근 명령은 거부
- 환경변수나 시크릿을 메시지에 포함하지 않음
```

4. 팀 스킬 생성

자주 사용하는 작업을 스킬로 만듭니다.

```
> 다음 스킬을 만들어줘:
> 이름: deploy-check
> 내용: 배포 전 체크리스트 - 테스트 통과 여부, 린트 통과 여부,
>       환경변수 설정 확인, DB 마이그레이션 확인
```

팀에서 유용한 스킬 예시:

- **deploy-check**: 배포 전 체크리스트 실행
- **pr-review**: PR URL을 받아 코드 리뷰 수행
- **incident-report**: 장애 보고서 템플릿 생성
- **onboarding**: 새 팀원용 환경 설정 가이드 안내

5. 퍼스널리티 설정 (SOUL.md)

팀 봇에 적합한 성격을 설정합니다.

```
# SOUL.md

## 성격
- 간결하고 직접적으로 답변
- 기술적 질문에는 코드 예시를 포함
- 모르는 것은 솔직히 모른다고 말함
- 팀 내부 용어를 자연스럽게 사용

## 톤
- 캐주얼하지만 정확한 톤
- 이모지 최소화
```

6. 게이트웨이 실행

```
hermes gateway
```

팀 사용 시나리오

코드 리뷰 요청

```
팀원: 이 PR 좀 봐줘 https://github.com/our/repo/pull/42
봇: PR #42 분석 결과:
    ✅ 테스트 통과
    ⚠️ src/api/handler.ts:23 - SQL 쿼리에 파라미터 바인딩 없음
    ⚠️ src/utils/parse.ts:15 - 에러 처리 누락
    💡 제안: handler.ts에서 prepared statement 사용 권장
```

문서 검색

```
팀원: 우리 API에서 인증 토큰 갱신하는 방법이 뭐야?
봇: docs/auth.md에 따르면, refresh token으로 POST /auth/refresh
    요청을 보내면 됩니다. 토큰 만료 시간은 기본 1시간입니다.
    (파일 경로: docs/auth.md#token-refresh)
```

배포 전 점검

```
팀원: deploy-check
봇: 배포 전 체크리스트:
    ✅ 모든 테스트 통과 (142/142)
    ✅ 린트 에러 없음
    ⚠️ .env.production에 NEW_API_KEY 미설정
    ✅ DB 마이그레이션 최신 상태
    → NEW_API_KEY 설정 후 배포 가능합니다.
```

보안 고려사항

팀 봇 운영 시 반드시 확인해야 할 사항:

- **컨테이너 백엔드 사용**: `terminal.backend: docker` 또는 `ssh`로 설정하여 호스트 시스템을 보호
- **DISCORD_ALLOWED_USERS 필수 설정**: 미설정 시 모든 요청이 거부됩니다. `GATEWAY_ALLOW_ALL_USERS`를 `true`로 설정하면 인증 없이 누구나 접근 가능하므로 절대 금지
- **승인 모드**: `approvals.mode: manual` (위험한 명령 실행 전 확인). 값은 `manual` · `smart` · `off`이며 기본은 `manual` 입니다
- **리소스 제한 설정**: CPU, 메모리 제한으로 남용 방지
- **정기 감사**: 메모리, 스킬, 크론 작업을 주기적으로 점검

컨테이너 백엔드 필수

팀 봇에서는 반드시 컨테이너 백엔드를 사용하세요. 봇이 실행하는 명령이 호스트 시스템에 직접 영향을 주면 보안 사고로 이어질 수 있습니다. Docker 백엔드 설정은 11장을 참조하세요.

운영 팁

로그 모니터링

팀 봇의 대화 내역과 도구 실행 로그를 주기적으로 확인합니다.

```
# 최근 세션 확인
hermes sessions list

# 특정 세션 보기(대화형 선택·검색)
hermes sessions browse

# 특정 세션을 파일로 내보내기
hermes sessions export --session-id <session_id> -
```

스킬 버전 관리

팀 스킬은 Git으로 관리하면 변경 이력을 추적할 수 있습니다.

```
cd ~/.hermes/skills
git init
git add .
git commit -m "팀 스킬 초기 설정"
```

사용량 관리

API 기반 LLM을 사용할 경우, 팀 봇의 사용량을 모니터링하세요. Ollama 로컬 모델을 사용하면 비용 걱정 없이 운영할 수 있습니다.

13-4. Mac Mini 홈 인텔리전스 허브

> ⚠️ ***부분 검증**: 이 페이지의 명령·설정·스킬 정보는 v0.16.0 소스 기준으로 재확인했습니다(Apple 스킬 설치 명령, HomeAssistant 환경 변수·도구, `hermes gateway` , 크론). 다만 Apple 미리알림·iMessage·메모·Find My, HomeAssistant 연동, Telegram 원격 제어를 함께 돌려 보는 통합 테스트는 iCloud 로그인 Mac과 HomeAssistant 인스턴스가 필요해 현재 테스트 환경에서는 수행하지 못했습니다.*

- 프로젝트: Mac Mini 홈 인텔리전스 허브
- 목표
- 아키텍처
- 1단계: Apple 스킬 설정
  - Homebrew와 CLI 도구 설치
  - 미리알림 테스트
  - iMessage 테스트
- 2단계: HomeAssistant 연동
  - 설정
  - 사용 예시
- 3단계: 크론으로 자동화
  - 모닝 브리핑
  - 야간 루틴
- 4단계: Telegram 원격 제어
- 이 프로젝트의 의미

프로젝트: Mac Mini 홈 인텔리전스 허브

이 프로젝트에서는 Mac Mini(또는 항상 켜 놓는 macOS 기기)에서 Hermes Agent를 실행하여, Apple 생태계와 스마트홈을 통합 관리하는 개인 AI 비서를 구축합니다.

목표

- Apple 미리알림, 메모, iMessage를 에이전트로 제어
- HomeAssistant 연동으로 스마트홈 기기 제어
- 크론으로 모닝 브리핑, 야간 루틴 자동화
- Telegram으로 외출 중에도 에이전트와 소통

아키텍처

```
iPhone/iPad ←(iCloud 동기화)→ Mac Mini (Hermes Agent)
                                    ↕
                              Ollama LLM 서버
                                    ↕
Telegram ←(메시징 게이트웨이)→ HomeAssistant
```

1단계: Apple 스킬 설정

Homebrew와 CLI 도구 설치

Apple 스킬 중 3개는 Homebrew로 설치하는 CLI 도구가 필요합니다(7장 참조). 테스트 Mac에는 Homebrew가 이미 설치되어 있으므로 CLI 도구만 설치하면 됩니다. Homebrew가 없는 환경이라면 brew.sh에서 먼저 설치하세요.

```
# Apple 스킬 CLI 도구 설치
brew install steipete/tap/remindctl    # 미리알림 (apple-reminders 스킬)
brew install steipete/tap/imsg         # iMessage (imessage 스킬)
brew install antoniorodr/memo/memo     # Apple 메모 (apple-notes 스킬)
```

미리알림 테스트

```
> 내일 오전 10시에 "치과 예약" 미리알림을 추가해줘
> 이번 주 미리알림 목록을 보여줘
> "장보기" 리스트에 우유, 빵, 계란을 추가해줘
```

iCloud를 통해 iPhone의 미리알림 앱에도 즉시 동기화됩니다.

iMessage 테스트

```
> 최근 iMessage 목록을 보여줘
> 아내에게 "오늘 늦을 것 같아" 메시지 보내줘
```

전체 디스크 접근 권한

iMessage 스킬을 사용하려면 터미널 앱에 **시스템 설정 > 개인정보 보호 및 보안 > 전체 디스크 접근** 권한을 부여해야 합니다. findmy 스킬은 별도 CLI 도구 없이 AppleScript로 Find My 앱 화면을 캡처한 뒤 멀티모달 LLM으로 읽는 방

식을 사용합니다(7장 참조).

2단계: HomeAssistant 연동

HomeAssistant가 설치되어 있다면, Hermes Agent의 내장 HomeAssistant 도구로 연결할 수 있습니다.

설정

```
# ~/.hermes/.env에 추가
HASS_TOKEN=your_long_lived_access_token
HASS_URL=http://homeassistant.local:8123
```

사용 예시

```
> 거실 조명을 켜줘
> 현재 실내 온도가 몇 도야?
> 현관문 잠금 상태를 확인해줘
```

3단계: 크론으로 자동화

모닝 브리핑

```
> 매일 아침 7시에 다음을 실행해줘:
> 1. 오늘 미리알림 목록 확인
> 2. 오늘 날씨 검색
> 3. 요약해서 Telegram으로 보내줘
```

야간 루틴

```
> 매일 밤 11시에 다음을 실행해줘:
> 1. 모든 조명 끄기
> 2. 현관문 잠금 확인
> 3. 내일 미리알림 미리 알려주기
```

4단계: Telegram 원격 제어

외출 중에도 Telegram으로 에이전트와 소통할 수 있습니다 (9장 참조).

```
> hermes gateway
```

Telegram에서:

```
장보기 리스트에 두부 추가해줘
거실 에어컨 25도로 설정해줘
아이 AirTag 위치 확인해줘     # findmy 스킬: AppleScript + 비전 LLM 방식
```

이 프로젝트의 의미

이 구성은 OpenClaw의 네이티브 iOS 앱이 제공하는 경험과 다른 방식으로 같은 목표를 달성합니다. OpenClaw는 앱으로, Hermes Agent는 시스템 레벨 접근으로 Apple 생태계와 통합합니다. Hermes Agent만의 장점은:

- **FindMy 통합**: OpenClaw에 없는 기능 (AppleScript + 비전 LLM 방식)
- **HomeAssistant 내장 도구**: 4개의 전용 도구로 스마트홈 직접 제어
- **자기 개선**: 반복 사용 시 자동으로 스킬 생성 (예: "야간 루틴" 스킬)
- **비용**: 로컬 LLM 사용 시 API 비용 없음